

✓ Algoritmos - Actividad Guiada 1

Nombre: Irune Urrutia Andres

URL:

[https://colab.research.google.com/drive/1ZeP1SAEY8_BEencNT0BfcLKPOpxoiGyhg#scrollTo=8TMvgNzg3mlo\\$0](https://colab.research.google.com/drive/1ZeP1SAEY8_BEencNT0BfcLKPOpxoiGyhg#scrollTo=8TMvgNzg3mlo$0)

http:https://github.com/iruneurrutia/O3MIAR_ALGORITMOS_ACTIVIDADES

✓ Torres de Hanoi con Divide y vencerás

```
def Torres_Hanoi(N, desde, hasta):  
    if N ==1 :  
        print("Lleva la ficha " ,desde , " hasta ", hasta )  
  
    else:  
        #Torres_Hanoi(N-1, desde, 6-desde-hasta )  
        Torres_Hanoi(N-1, desde, 6-desde-hasta )  
        print("Lleva la ficha " ,desde , " hasta ", hasta )  
        #Torres_Hanoi(N-1,6-desde-hasta, hasta )  
        Torres_Hanoi(N-1, 6-desde-hasta , hasta )  
  
Torres_Hanoi(3, 1 , 3)
```

```
Lleva la ficha 1 hasta 3  
Lleva la ficha 1 hasta 2  
Lleva la ficha 3 hasta 2  
Lleva la ficha 1 hasta 3  
Lleva la ficha 2 hasta 1  
Lleva la ficha 2 hasta 3  
Lleva la ficha 1 hasta 3
```

[verde , amarillo, azul] --- [] ---- [] // verde < amarilla < azul

Torres_Hanoi(3, 1, 3) - Torres_Hanoi(2, 1, 2) - Torres_Hanoi(1, 1, 3)

Lleva la ficha de 1 hasta 3 [amarillo, azul] --- [] ---- [verde]

Torres_Hanoi(3, 1, 3) - Torres_Hanoi(2, 1, 2)

Lleva la ficha 1 hasta 2 [azul] --- [amarillo] ---- [verde]

Torres_Hanoi(3, 1, 3) - Torres_Hanoi(2, 1, 2)

Torres_Hanoi(3, 1, 3) - Torres_Hanoi(2, 1, 2) - Torres_Hanoi(1, 3, 2)

Lleva la ficha 3 hasta 2 [azul] --- [verde, amarillo] ---- []

Torres_Hanoi(3, 1, 3)

Lleva la ficha 1 hasta 3 [] --- [verde, amarillo] ---- [azul]

Torres_Hanoi(3, 1, 3) - Torres_Hanoi(2, 2, 3) [verde] --- [] ---- [amarillo, azul] [] --
- [] ---- [verde, amarillo, azul]

desde 1, hasta 3 --- $6 - 1 - 3 = 2$

desde 1, hasta 2 --- $6 - 1 - 2 = 3$

desde 3, hasta 2 --- $6 - 3 - 2 = 5$

```
#Sucesión_de_Fibonacci
#https://es.wikipedia.org/wiki/Sucesi%C3%B3n_de_Fibonacci
#Calculo del termino n-simo de la suscesión de Fibonacci
def Fibonacci(N:int):
    if N < 2:
        return 1
    else:
        return Fibonacci(N-1)+Fibonacci(N-2)

Fibonacci(5)
```

8

```
F(1) =1
F(2) = 1
F(3) = 2
F(4) = 3 = F(3) + F(2)
F(5) = 5 = F(4) + F(3)
F(6) = 8 = F(5) + F(4)
```

✓ Devolución de cambio por técnica voraz

```
def cambio_monedas(N, SM):
    SOLUCION = [0]*len(SM)    #SOLUCION = [0,0,0,0,..]
    ValorAcumulado = 0

    for i,valor in enumerate(SM): # enumerar
        monedas = (N-ValorAcumulado)//valor
        SOLUCION[i] = monedas
        ValorAcumulado = ValorAcumulado + monedas*valor

    if ValorAcumulado == N:
        return SOLUCION

cambio_monedas(15, [25,10,5,1])

[0, 1, 1, 0]
```

✓ N-Reinas por técnica de vueta atrás

```
def escribe(S):
    n = len(S)
    for x in range(n):
        print("")
        for i in range(n):
            if S[i] == x+1:
                print(" X " , end="")
            else:
                print(" - ", end="")

def es_prometedora(SOLUCION,etapa):
    #print(SOLUCION)
    #Si la solución tiene dos valores iguales no es valida => Dos rei
    for i in range(etapa+1):
        #print("El valor " + str(SOLUCION[i]) + " está " + str(SOLUCION[i]))
        if SOLUCION.count(SOLUCION[i]) > 1:
            return False

    #Verifica las diagonales
    for j in range(i+1, etapa +1 ):
        #print("Comprobando diagonal de " + str(i) + " y " + str(j))
        if abs(i-j) == abs(SOLUCION[i]-SOLUCION[j]) : return False
    return True
```

```
def reinas(N, solucion=[], etapa=0):
    if len(solucion) == 0:
        solucion=[0 for i in range(N)]

    for i in range(1, N+1):
        solucion[etapa] = i

        if es_prometedora(solucion, etapa):
            if etapa == N-1:
                print(solucion)
                #escribe(solucion)
                print()
            else:
                reinas(N, solucion, etapa+1)
        else:
            None

    solucion[etapa] = 0
```

```
reinas(8)
```

```
[1, 5, 8, 6, 3, 7, 2, 4]
```

```
[1, 6, 8, 3, 7, 4, 2, 5]
```

```
[1, 7, 4, 6, 8, 2, 5, 3]
```

```
[1, 7, 5, 8, 2, 4, 6, 3]
```

```
[2, 4, 6, 8, 3, 1, 7, 5]
```

```
[2, 5, 7, 1, 3, 8, 6, 4]
```

```
[2, 5, 7, 4, 1, 8, 6, 3]
```

```
[2, 6, 1, 7, 4, 8, 3, 5]
```

```
[2, 6, 8, 3, 1, 4, 7, 5]
```

```
[2, 7, 3, 6, 8, 5, 1, 4]
```

```
[2, 7, 5, 8, 1, 4, 6, 3]
```

```
[2, 8, 6, 1, 3, 5, 7, 4]
```

```
[3, 1, 7, 5, 8, 2, 4, 6]
```

```
[3, 5, 2, 8, 1, 7, 4, 6]
```

```
[3, 5, 2, 8, 6, 4, 7, 1]
```

```
[3, 5, 7, 1, 4, 2, 8, 6]
```

```

[3, 5, 8, 4, 1, 7, 2, 6]
[3, 6, 2, 5, 8, 1, 7, 4]
[3, 6, 2, 7, 1, 4, 8, 5]
[3, 6, 2, 7, 5, 1, 8, 4]
[3, 6, 4, 1, 8, 5, 7, 2]
[3, 6, 4, 2, 8, 5, 7, 1]
[3, 6, 8, 1, 4, 7, 5, 2]
[3, 6, 8, 1, 5, 7, 2, 4]
[3, 6, 8, 2, 4, 1, 7, 5]
[3, 7, 2, 8, 5, 1, 4, 6]
[3, 7, 2, 8, 6, 4, 1, 5]
[3, 8, 4, 7, 1, 6, 2, 5]
[4, 1, 5, 8, 2, 7, 3, 6]

```

✓ Viaje por el río. Programación dinámica

```

TARIFAS = [
    [0,5,4,3,999,999,999],
    [999,0,999,2,3,999,11],
    [999,999, 0,1,999,4,10],
    [999,999,999, 0,5,6,9],
    [999,999, 999,999,0,999,4],
    [999,999, 999,999,999,0,3],
    [999,999,999,999,999,999,0]
]

#####
def Precios(TARIFAS):
    #####
    #Total de Nodos
    N = len(TARIFAS[0])

    #Inicialización de la tabla de precios
    PRECIOS = [ [9999]*N for i in [9999]*N]
    RUTA = [ [""]*N for i in [""]*N]

    for i in range(0,N-1):

```

```

    RUTA[i][i] = i                                #Para ir de i a i se "pasa por i"
    PRECIOS[i][i] = 0                             #Para ir de i a i se se paga 0
    for j in range(i+1, N):
        MIN = TARIFAS[i][j]
        RUTA[i][j] = i

        for k in range(i, j):
            if PRECIOS[i][k] + TARIFAS[k][j] < MIN:
                MIN = min(MIN, PRECIOS[i][k] + TARIFAS[k][j] )
                RUTA[i][j] = k                    #Anota que para ir de i a j hay
                PRECIOS[i][j] = MIN

    return PRECIOS,RUTA

#####

PRECIOS,RUTA = Precios(TARIFAS)
#print(PRECIOS[0][6])

print("PRECIOS")
for i in range(len(TARIFAS)):
    print(PRECIOS[i])

print("\nRUTA")
for i in range(len(TARIFAS)):
    print(RUTA[i])

#Determinar la ruta con Recursividad
def calcular_ruta(RUTA, desde, hasta):
    if desde == hasta:
        #print("Ir a :" + str(desde))
        return ""
    else:
        return str(calcular_ruta( RUTA, desde, RUTA[desde][hasta])) +
                ',' + \
                str(RUTA[desde][hasta] \
                )

print("\nLa ruta es:")
calcular_ruta(RUTA, 0,6)

```

```

PRECIOS
[0, 5, 4, 3, 8, 8, 11]
[9999, 0, 999, 2, 3, 8, 7]
[9999, 9999, 0, 1, 6, 4, 7]
[9999, 9999, 9999, 0, 5, 6, 9]
[9999, 9999, 9999, 9999, 0, 999, 4]
[9999, 9999, 9999, 9999, 9999, 0, 3]
[9999, 9999, 9999, 9999, 9999, 9999, 9999]

```

RUTA

```
[0, 0, 0, 0, 1, 2, 5]
['', 1, 1, 1, 1, 3, 4]
['', '', 2, 2, 3, 2, 5]
['', '', '', 3, 3, 3, 3]
['', '', '', '', 4, 4, 4]
['', '', '', '', '', 5, 5]
```

✓ EJERCICIOS ADICIONALES

Problema: Encontrar los dos puntos más cercanos

- Dado un conjunto de puntos se trata de encontrar los dos puntos más cercanos
- Guía para aprendizaje:
 - Suponer en 1D, o sea, una lista de números: [3403, 4537, 9089, 9746, 7259,
 - Primer intento: Fuerza bruta
 - Calcular la complejidad. ¿Se puede mejorar?
 - Segundo intento. Aplicar Divide y Vencerás
 - Calcular la complejidad. ¿Se puede mejorar?
 - Extender el algoritmo a 2D: [(1122, 6175), (135, 4076), (7296, 2741)...
 - Extender el algoritmo a 3D.

```
import math
import random
```

```

def dist1d(a, b):
    return abs(a - b)

def closest_pair_1d_brute(points):
    """
    Comparamos todas las parejas posibles.
    Complejidad:  $O(n^2)$   $\rightarrow$   $n*(n-1)/2$  comparaciones
    """
    mejor_dist = math.inf
    mejor_par = None
    for i in range(len(points)):
        for j in range(i + 1, len(points) ):
            d = dist1d(points[i], points[j])
            if d < mejor_dist:
                mejor_dist = d
                mejor_par = (points[i], points[j])
    return mejor_par, mejor_dist

def closest_pair_1d_dyv(points):
    """
    Ordenamos los puntos.
    El par mas cercano siempre está formado por elementos consecutivo
    Complejidad :  $O(n \log n)$ 
    """

    pts = sorted(points)
    mejor_dist = math.inf
    mejor_par = None
    for i in range(len(pts) - 1):
        d = dist1d(pts[i], pts[i + 1])
        if d < mejor_dist:
            mejor_dist = d
            mejor_par = (pts[i], pts[i + 1])

    return mejor_par, mejor_dist

```

```

def dist2d(p, q):
    return math.sqrt((p[0] - q[0])**2 + (p[1] - q[1])**2)

def closest_pair_2d_brute(points):
    """Fuerza bruta: Complejidad:  $O(n^2)$ ."""
    mejor_dist = math.inf
    mejor_par = None
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = dist2d(points[i], points[j])
            if d < mejor_dist:
                mejor_dist = d
                mejor_par = (points[i], points[j])
    return mejor_par, mejor_dist

```

```

def closest_pair_2d_dyv(points):
    """
    Divide y Vencerás.
    Complejidad:  $O(n \log n)$ 
    """
    if len(points) < 2:
        return None, math.inf

    px = sorted(points, key=lambda p: p[0]) # Ordenamos los puntos
    py = sorted(points, key=lambda p: p[1]) # Ordenamos los puntos

    def brute(pts):
        mejor_dist = math.inf
        mejor_par = None

        for i in range(len(pts)):
            for j in range(i + 1, len(pts)):
                d = dist2d(pts[i], pts[j])
                if d < mejor_dist:
                    mejor_dist, mejor_par = d, (pts[i], pts[j])
        return mejor_par, mejor_dist

    def rec(px, py):
        n = len(px)
        if n <= 3:
            return brute(px)

        mid = n // 2 # Dividimos el conjunto por la mitad
        punto_medio = px[mid]

        px_izq, px_der = px[:mid], px[mid:]
        # usamos coordenada x del punto de corte para repartir py
        # sin volver a ordenar
        py_izq, py_der = [], []
        izq_ids = set(id(p) for p in px_izq)

```

```

    for p in py:
        if id(p) in izq_ids: # Punto que pertenece a la mitad i
            py_izq.append(p)
        else:
            py_der.append(p)

    par_izq, d_izq = rec(px_izq, py_izq)
    par_der, d_der = rec(px_der, py_der)

    if d_izq <= d_der:
        mejor_par, delta = par_izq, d_izq
    else:
        mejor_par, delta = par_der, d_der

    # Creamos una franja alrededor de la linea de división

    franja = [p for p in py if abs(p[0] - punto_medio[0]) < del

    m = len(franja)
    for i in range(m):
        # basta comparar con los siguientes 7 puntos (propiedad
        for j in range(i + 1, min(i + 8, m)):
            d = dist2d(franja[i], franja[j])
            if d < delta:
                delta = d
                mejor_par = (franja[i], franja[j])

    return mejor_par, delta

return rec(px, py)

```

```

def dist3d(p, q):
    return math.sqrt((p[0]-q[0])**2 + (p[1]-q[1])**2 + (p[2]-q[2])**2)

def closest_pair_3d_brute(points):
    """Fuerza bruta. Complejidad: O(n^2)."""
    mejor_dist = math.inf
    mejor_par = None
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = dist3d(points[i], points[j])
            if d < mejor_dist:
                mejor_dist = d
                mejor_par = (points[i], points[j])
    return mejor_par, mejor_dist

def closest_pair_3d_dyv(points):
    """

```

Extensión del mismo esquema de Divide y Vencerás a 3D.

Complejidad: $O(n \log n)$

.....

```
if len(points) < 2:
    return None, math.inf

px = sorted(points, key=lambda p: p[0])
py = sorted(points, key=lambda p: p[1]) # ordenamos por Y para

def brute(pts):
    mejor_dist = math.inf
    mejor_par = None
    for i in range(len(pts)):
        for j in range(i + 1, len(pts)):
            d = dist3d(pts[i], pts[j])
            if d < mejor_dist:
                mejor_dist, mejor_par = d, (pts[i], pts[j])
    return mejor_par, mejor_dist

def rec(px, py):
    n = len(px)
    if n <= 3:
        return brute(px)

    mid = n // 2
    punto_medio = px[mid]

    px_izq, px_der = px[:mid], px[mid:]
    izq_ids = set(id(p) for p in px_izq)
    # Separamos py sin volver a ordenar
    py_izq, py_der = [], []
    for p in py:
        if id(p) in izq_ids:
            py_izq.append(p)
        else:
            py_der.append(p)

    # Resolvemos cada mitad
    par_izq, d_izq = rec(px_izq, py_izq)
    par_der, d_der = rec(px_der, py_der)

    # Nos quedamos con la menor distancia
    if d_izq <= d_der:
        mejor_par, delta = par_izq, d_izq
    else:
        mejor_par, delta = par_der, d_der

    # Franja alrededor del plano de división
    franja = [p for p in py if abs(p[0] - punto_medio[0]) < delt

    # Comparamos los puntos de la franja
```

```

        for i in range(len(franja)):
            for j in range(i + 1, len(franja)):
                d = dist3d(franja[i], franja[j])

                if d < delta:
                    delta = d
                    mejor_par = (franja[i], franja[j])

            return mejor_par, delta

    return rec(px, py)

```

✓ CASO DE USO

```

N = 1000

LISTA_1D = [
    random.randrange(1, 10000)
    for _ in range(N)
]

LISTA_2D = [
    (random.randrange(1, 10000),
     random.randrange(1, 10000))
    for _ in range(N)
]

LISTA_3D = [
    (random.randrange(1, 10000),
     random.randrange(1, 10000),
     random.randrange(1, 10000))
    for _ in range(N)
]

```

```
print("1D")
print("Brute:", closest_pair_1d_brute(LISTA_1D))
print("D&V:", closest_pair_1d_dyv(LISTA_1D))
```

```
print("\n2D")
print("Brute:", closest_pair_2d_brute(LISTA_2D))
print("D&V:", closest_pair_2d_dyv(LISTA_2D))
```

```
print("\n3D")
print("Brute:", closest_pair_3d_brute(LISTA_3D))
print("D&V:", closest_pair_3d_dyv(LISTA_3D))
```

```
1D
Brute: ((9983, 9983), 0)
D&V: ((9, 9), 0)
```

```
2D
Brute: (((4575, 5713), (4584, 5710)), 9.486832980505138)
D&V: (((4584, 5710), (4575, 5713)), 9.486832980505138)
```

```
3D
Brute: (((5398, 9103, 7917), (5349, 9084, 7867)), 72.53964433328854)
D&V: (((5349, 9084, 7867), (5398, 9103, 7917)), 72.53964433328854)
```